

INPUT AND OUTPUT

Get structured output from agents

Return validated JSON from agent workflows using JSON Schema, Zod, or Pydantic. Get type-safe, structured data after multi-turn tool use.

Structured outputs let you define the exact shape of data you want back from an agent. The agent can use any tools it needs to complete the task, and you still get validated JSON matching your schema at the end. Define a **JSON Schema** for the structure you need, and the SDK validates the output against it, re-prompting on mismatch. If validation does not succeed within the retry limit, the result is an error instead of structured data; see **Error handling**.

For full type safety, use **Zod** (TypeScript) or **Pydantic** (Python) to define your schema and get strongly-typed objects back.

Why structured outputs?

Agents return free-form text by default, which works for chat but not when you need to use the output programmatically. Structured outputs give you typed data you can pass directly to your application logic, database, or UI components.

Consider a recipe app where an agent searches the web and brings back recipes. Without structured outputs, you get free-form text that you'd need to parse yourself. With structured outputs, you define the shape you want and get typed data you can use directly in your app.

▸ Without structured outputs

▸ With structured outputs

Quick start

To use structured outputs, define a **JSON Schema** describing the shape of data you want, then pass it to `query()` via the `outputFormat` option (TypeScript) or `output_format` option (Python).

When the agent finishes, the result message includes a `structured_output` field with validated data matching your schema.

The example below asks the agent to research Anthropic and return the company name, year founded, and headquarters as structured output.

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Define the shape of data you want back
const schema = {
  type: "object",
  properties: {
    company_name: { type: "string" },
    founded_year: { type: "number" },
    headquarters: { type: "string" }
  },
  required: ["company_name"]
};

for await (const message of query({
  prompt: "Research Anthropic and provide key company information",
  options: {
    outputFormat: {
      type: "json_schema",
      schema: schema
    }
  }
})) {
  // The result message contains structured_output with validated data
  if (message.type === "result" && message.subtype === "success" &&
message.structured_output) {
    console.log(message.structured_output);
    // { company_name: "Anthropic", founded_year: 2021, headquarters: "San
Francisco, CA" }
  }
}
```

Type-safe schemas with Zod and Pydantic

Instead of writing JSON Schema by hand, you can use **Zod** (TypeScript) or **Pydantic** (Python) to define your schema. These libraries generate the JSON Schema for you and let you parse the response into a fully-typed object you can use throughout your codebase with autocomplete and

type checking.

The example below defines a schema for a feature implementation plan with a summary, list of steps (each with complexity level), and potential risks. The agent plans the feature and returns a typed `FeaturePlan` object. You can then access properties like `plan.summary` and iterate over `plan.steps` with full type safety.

```

import { z } from "zod";
import { query } from "@anthropic-ai/claude-agent-sdk";

// Define schema with Zod
const FeaturePlan = z.object({
  feature_name: z.string(),
  summary: z.string(),
  steps: z.array(
    z.object({
      step_number: z.number(),
      description: z.string(),
      estimated_complexity: z.enum(["low", "medium", "high"])
    })
  ),
  risks: z.array(z.string())
});

type FeaturePlan = z.infer<typeof FeaturePlan>;

// Convert to JSON Schema
const schema = z.toJSONSchema(FeaturePlan);

// Use in query
for await (const message of query({
  prompt:
    "Plan how to add dark mode support to a React app. Break it into implementation steps.",
  options: {
    outputFormat: {
      type: "json_schema",
      schema: schema
    }
  }
})) {
  if (message.type === "result" && message.subtype === "success" &&
message.structured_output) {
    // Validate and get fully typed result
    const parsed = FeaturePlan.safeParse(message.structured_output);
    if (parsed.success) {
      const plan: FeaturePlan = parsed.data;
      console.log(`Feature: ${plan.feature_name}`);
      console.log(`Summary: ${plan.summary}`);
      plan.steps.forEach((step) => {
        console.log(`${step.step_number}. [${step.estimated_complexity}]

```

```
${step.description}`);
    });
  }
}
}
```

Benefits:

- Full type inference (TypeScript) and type hints (Python)
- Runtime validation with `safeParse()` or `model_validate()`
- Better error messages
- Composable, reusable schemas

Output format configuration

The `outputFormat` (TypeScript) or `output_format` (Python) option accepts an object with:

- `type` : Set to `"json_schema"` for structured outputs
- `schema` : A **JSON Schema** object defining your output structure. You can generate this from a Zod schema with `z.toJSONSchema()` or a Pydantic model with `.model_json_schema()`

The SDK supports standard JSON Schema features including all basic types (object, array, string, number, boolean, null), `enum`, `const`, `required`, nested objects, and `$ref` definitions. For the full list of supported features and limitations, see [JSON Schema limitations](#).

Example: TODO tracking agent

This example demonstrates how structured outputs work with multi-step tool use. The agent needs to find TODO comments in the codebase, then look up git blame information for each one. It autonomously decides which tools to use (Grep to search, Bash to run git commands) and combines the results into a single structured response.

The schema includes optional fields (`author` and `date`) since git blame information might not be available for all files. The agent fills in what it can find and omits the rest.

```

import { query } from "@anthropic-ai/claude-agent-sdk";

// Define structure for TODO extraction
const todoSchema = {
  type: "object",
  properties: {
    todos: {
      type: "array",
      items: {
        type: "object",
        properties: {
          text: { type: "string" },
          file: { type: "string" },
          line: { type: "number" },
          author: { type: "string" },
          date: { type: "string" }
        },
        required: ["text", "file", "line"]
      },
    },
    total_count: { type: "number" }
  },
  required: ["todos", "total_count"]
};

// Agent uses Grep to find TODOs, Bash to get git blame info
for await (const message of query({
  prompt: "Find all TODO comments in this codebase and identify who added them",
  options: {
    outputFormat: {
      type: "json_schema",
      schema: todoSchema
    }
  }
})) {
  if (message.type === "result" && message.subtype === "success" &&
message.structured_output) {
    const data = message.structured_output as { total_count: number; todos:
Array<{ file: string; line: number; text: string; author?: string; date?:
string }> };
    console.log(`Found ${data.total_count} TODOs`);
    data.todos.forEach((todo) => {
      console.log(`${todo.file}:${todo.line} - ${todo.text}`);
    });
  }
}

```

```
    if (todo.author) {
      console.log('  Added by ${todo.author} on ${todo.date}');
    }
  });
}
```

Error handling

Structured output generation can fail when the agent cannot produce valid JSON matching your schema. This typically happens when the schema is too complex for the task, the task itself is ambiguous, or the agent hits its retry limit trying to fix validation errors. It can also happen without any validation failure: a **model fallback** can retract an already-completed output mid-stream, and if no retry replaces it the run ends with the same error. Check the `errors` field on the result message to tell the two causes apart before debugging your schema.

When an error occurs, the result message has a `subtype` indicating what went wrong:

Subtype	Meaning
<code>success</code>	Output was generated and validated successfully
<code>error_max_structured_output_retries</code>	No valid output remained after multiple attempts (validation failures, or a model-fallback retraction with no successful retry)

The example below checks the `subtype` field to determine whether the output was generated successfully or if you need to handle a failure:

```

for await (const msg of query({
  prompt: "Extract contact info from the document",
  options: {
    outputFormat: {
      type: "json_schema",
      schema: contactSchema
    }
  }
})) {
  if (msg.type === "result") {
    if (msg.subtype === "success" && msg.structured_output) {
      // Use the validated output
      console.log(msg.structured_output);
    } else if (msg.subtype === "error_max_structured_output_retries") {
      // Handle the failure - retry with simpler prompt, fall back to
      unstructured, etc.
      console.error("Could not produce valid output");
    }
  }
}
}

```

Tips for avoiding errors:

- **Keep schemas focused.** Deeply nested schemas with many required fields are harder to satisfy. Start simple and add complexity as needed.
- **Match schema to task.** If the task might not have all the information your schema requires, make those fields optional.
- **Use clear prompts.** Ambiguous prompts make it harder for the agent to know what output to produce.

Related resources

- **JSON Schema documentation:** learn JSON Schema syntax for defining complex schemas with nested objects, arrays, enums, and validation constraints
- **API Structured Outputs:** use structured outputs with the Claude API directly for single-turn requests without tool use
- **Custom tools:** give your agent custom tools to call during execution before returning structured output

